RESEARCH & PROTOTYPE REVIEW

Prediction Process Report

How current RUL displays work in source, which claims are unverified and how to build a real pipeline.

SIH 2026 | PS SIH26250 | Alchemy Architects

SOURCE-CODE REVIEW + PUBLIC RESEARCH | 20 OCTOBER 2026

ALCHEMY ARCHITECTS

Purpose

This report documents the prediction path visible in the supplied AAKAASH ADHINAYAK source export and separates currently implemented displays from a recommended reproducible prediction pipeline. The distinction is essential because the interface presents candidate model performance figures that are not derived by a training/evaluation workflow found in the inspected code.

Current verdict: the exported UI displays Remaining Useful Life (RUL) estimates in a seeded table. The inspected project files do not show dataset parsing, model fitting, model serialization, inference from a fitted model, or calculation of displayed RMSE/MAE/interval metrics. Do not present the current screen as a verified trained predictor.

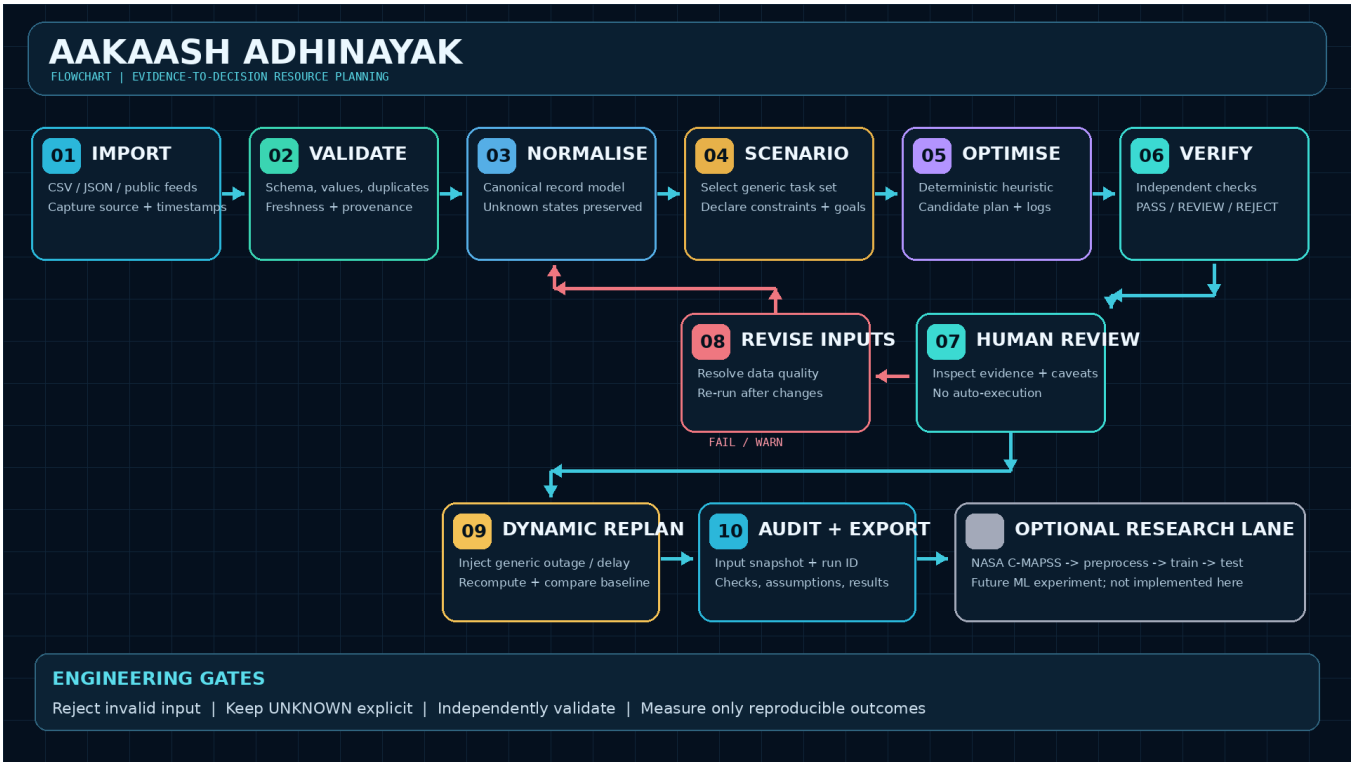
1. Current application path

Step	What the supplied source does	Status
1. Load sample record list	`NASA_CMAPSS_RECORDS` is declared in `src/data/seedData.ts` with five pre-populated cycle rows for Unit 01. Sensor values, `trueRul`, baseline prediction and candidate prediction are stored as fields.	Seeded data, not demonstrated raw archive ingestion.
2. Render predictive lab	`PredictiveMaintenanceLab.tsx` iterates over supplied records and formats the `trueRul`/predicted fields and wear bands. No prediction method is called in the visible component.	UI table and rule-derived labels.
3. Render model evaluation	`ModelEvaluation.tsx` declares static sensor weights and hard-coded baseline/candidate metric objects. The export action writes those constants to JSON.	Static presentation values.
4. API metadata response	`server.ts` `/api/nasa-cmapss` returns dataset metadata, sensor descriptions and static RMSE/MAE/test-unit/interval values. The inspected route does not load or train a model.	Metadata/metrics endpoint, not inference.
5. Connector fallback	`dataConnector.ts` calls the local API but falls back to the same seed list when no usable API records are returned.	Fallback path; source must remain clearly labelled.

2. The metrics currently shown

Metric shown in export	Displayed value	Interpretation for PPT
Baseline RMSE	34.2 cycles	Unverified display constant; not independently recomputed from held-out predictions.
Baseline MAE	28.5 cycles	Unverified display constant.
Candidate RMSE	14.8 cycles	Unverified display constant; model training and predictions were not found.
Candidate MAE	10.6 cycles	Unverified display constant.
“Error improvement”	56.7%	Arithmetically follows from (34.2 - 14.8)/34.2, but the underlying error metrics themselves are not reproduced by the source.
95% uncertainty interval	+/- 12.4 cycles	Shown as a fixed string/value; interval construction/calibration not found.
Evaluation split	70% train / 15% validation / 15% test	Described in the UI/export function; no actual split execution/manifest was found.

3. Recommended reproducible prediction pipeline



The diagram's optional ML lane can be implemented separately from the planning scheduler. Use the exact C-MAPSS archive and citation supplied by NASA; do not treat simulated commercial turbofan data as military telemetry. The steps below are a proposal, not a description of code already implemented.

Stage	Implementation checklist	Evidence to retain
Dataset provenance	Download the exact PCoE C-MAPSS dataset. Capture filename, dataset split, retrieval date, source URL, terms/attribution and checksum.	README, checksum, version manifest.
Data parsing	Parse each engine unit trajectory and sensor channels; validate schema, ranges, duplicates and missing cycles.	Schema report + rejected rows.
Target	Define RUL target (e.g., cycles remaining to run-to-failure); decide and document target capping if used.	Target definition/version.
Leakage-safe split	Split on engine unit before fitting transformers; use training-only scaling and validation for hyperparameters.	Split manifest with unit IDs.
Baseline model	Implement a documented baseline such as constant/linear trend or a transparent statistical model.	Prediction file + RMSE/MAE computation.
Candidate model	Start with a small regressor; evaluate a deep sequence model only if the dataset and compute justify it.	Source, dependency versions, seed and model artifact.
Uncertainty	Use a documented method such as bootstrap or conformal intervals with calibration data and coverage results.	Coverage vs nominal 95% chart.
Integration	Return RUL, uncertainty, model/data version, computed-at time and a non-operational limitation label.	API test response + log.
Monitoring	Check distribution drift and invalid sensors; never silently infer from unknown input.	Data-quality alert and fallback state.

4. Metrics that should be reported

- MAE = mean absolute difference between actual and predicted RUL, in cycles.
- RMSE = square root of mean squared prediction error, in cycles; more sensitive to large misses.
- Report results per held-out engine unit or a defined group, then aggregate using a documented method.

- For an interval advertised as 95%, report empirical coverage and average width on held-out data; a fixed +/- value is not sufficient.
- Report baseline and candidate on exactly the same test split, with dataset version, preprocessing, code commit and run date.
- Prediction error on simulated commercial-engine data does not establish aircraft-specific or defence-use performance.

5. PPT wording recommendation

“The interface includes a predictive-maintenance experiment panel referencing NASA C-MAPSS RUL. In the current source export, the metrics and sample predictions are prefilled demonstration values rather than the result of a reproducible in-app model-training pipeline. Building and independently evaluating that pipeline is a defined next milestone.”

References and source notes

[1] NASA Prognostics Center of Excellence Dataset Repository
https://www.nasa.gov/intelligent-systems-division/discovery-and-systems-health/pcoe/pcoe-data-set-repository/
Official NASA repository lists turbofan engine degradation simulation data and the C-MAPSS benchmark. It is not military telemetry.

[2] NASA Dashlink - Turbofan Degradation Simulation Dataset
https://c3.ndc.nasa.gov/dashlink/resources/139/
Background description and citation for the simulated degradation benchmark.

[3] scikit-learn model evaluation documentation
https://scikit-learn.org/stable/modules/model_evaluation.html
Reference for reproducible regression evaluation metrics such as mean absolute error and root mean squared error.